

$O(n)$

ALGORİTMA ANALİZİ · ARAŞTIRMA RAPORU

Big O Notasyonu Nedir?

Tanımı, matematikteki kökeni, bilgisayar bilimine geçişi, kullanım alanları ve basit örneklerle açıklanan bir sunum hazırlık raporu.

01 Big O Notasyonu Nedir?

Big O notasyonu, bir algoritmanın girdi büyüklüğü arttıkça çalışma süresinin veya kullandığı belleğin nasıl büyüdüğünü tarif etmek için kullanılan matematiksel bir gösterimdir. Doğrudan "algoritma şu kadar saniyede çalışır" demez; bunun yerine girdi büyüklüğü (genellikle n ile gösterilir) büyüdükçe işlem sayısının hangi hızla arttığını, yani büyüme eğilimini tarif eder.

Bu nedenle Big O bir "üst sınır" ifadesidir: bir algoritmanın en kötü senaryoda ne kadar yavaşlayabileceğinin garantisini verir. Örneğin bir algoritma için $O(n)$ deniyorsa, girdi büyüklüğü iki katına çıktığında yapılan iş de kabaca iki katına çıkar; $O(n^2)$ deniyorsa girdi iki katına çıktığında iş yaklaşık dört katına çıkar.

Kısaca: Big O, donanımdan ve programlama dilinden bağımsız olarak, bir çözümün "büyüklük arttıkça ne kadar zorlanacağını" ölçmenin ortak dilidir.

Neden saniye yerine bu gösterim kullanılır?

Bir algoritmanın gerçek çalışma süresi; işlemcinin hızına, programlama diline, derleyiciye ve o an çalışan diğer programlara göre değişir. Aynı kod, bir bilgisayarda 2 saniyede, başka bir bilgisayarda 0,5 saniyede çalışabilir. Bu da saniye cinsinden ölçümü adil bir karşılaştırma aracı olmaktan çıkarır. Big O ise donanımdan bağımsız, yalnızca "adım sayısının girdiyle nasıl ölçeklendiğine" odaklanan bir soyutlama sunduğu için algoritmaları adil şekilde kıyaslamayı mümkün kılar.

02 Bu Fikir İlk Ne Zaman Ortaya Çıktı?

Big O notasyonunun kökeni, bilgisayar biliminden çok önce, saf matematiğe (sayılar teorisine) dayanır. Bilgisayar bilimine girişi ise ancak 20. yüzyılın ikinci yarısında, algoritma analizinin ayrı bir disiplin haline gelmesiyle gerçekleşmiştir.

- 1894** **Paul Bachmann**, "Analytische Zahlentheorie" (Analitik Sayılar Teorisi) adlı eserinin ikinci cildinde büyük O sembolünü ilk kez matematiksel bir gösterim olarak kullanır. "O" harfi, Almanca "Ordnung" (meritebe, derece) kelimesinden gelir.
- 1909** **Edmund Landau**, asal sayıların dağılımı üzerine yazdığı eserde Bachmann'ın notasyonunu benimser, yaygınlaştırır ve "küçük o" (little-o) gösterimini ekler. Bu yüzden literatürde bu aile "Bachmann-Landau notasyonu" olarak da anılır.
- 1914** **G. H. Hardy** ve **J. E. Littlewood**, bir çalışmalarında "büyük Omega" (Ω) gösterimini (alt sınır fikri) tanıtır.
- 1960'lar** Bilgisayar biliminin ayrı bir alan olarak olgunlaşmasıyla birlikte **Donald Knuth**, algoritmaların verimliliğini incelemeyi sistematik hale getirir; 1968'de yayımlanan *The Art of Computer Programming* serisinin ilk cildi bu alanın temel kaynaklarından biri olur.
- 1976** **Donald Knuth**, "Big Omicron and Big Omega and Big Theta" başlıklı makalesinde (ACM SIGACT News) O, Ω ve Θ notasyonlarını bilgisayar bilimi için netleştirir ve standartlaştırır; bu makale, notasyonun bilgisayar bilimindeki modern kullanımının temel taşı kabul edilir.

Dikkat edilmesi gereken nokta: Big O, "bilgisayar bilimi için" icat edilmiş bir kavram değildir. Önce matematikte (sayılar teorisinde) doğmuş, yaklaşık 70-80 yıl sonra Donald Knuth ve çağdaşları tarafından algoritma analizine uyarlanmıştır. Sunumda bu ayrımı netleştirmek izleyicinin kafa karışıklığını önler.

03 Temel Amacı Nedir?

Big O notasyonunun temel amacı, farklı algoritmaları veya çözümleri, donanımdan ve uygulama detaylarından bağımsız, ortak ve nesnel bir ölçütle karşılaştırmaktır. Bunu üç ana hedef altında özetlemek mümkündür:

1. Ölçeklenebilirliği tahmin etmek

Bir çözüm 100 kayıtla hızlı çalışıyor olabilir; asıl soru, 1 milyon veya 1 milyar kayıtla ne olacağıdır. Big O, bu büyümeyi küçük veri üzerinde test etmeden, matematiksel olarak öngörmeyi sağlar.

2. Algoritmaları kıyaslamak

Aynı problemi çözen iki farklı algoritma varsa (örneğin iki farklı sıralama yöntemi), hangisinin büyük veri kümelerinde daha avantajlı olacağını donanımdan bağımsız şekilde karşılaştırmayı sağlar.

3. Kaynak kullanımını iki boyutta değerlendirmek

Big O yalnızca zamanla ilgili değildir; iki ayrı eksenle kullanılır:

- **Zaman karmaşıklığı (time complexity):** Algoritmanın girdi büyüdükçe kaç "adım" attığının tahmini.
- **Uzay/alan karmaşıklığı (space complexity):** Algoritmanın girdi büyüdükçe ne kadar ek bellek kullandığının tahmini.

Önemli inceliği: Big O "en kötü durumu" (worst case) tarif eder. Bir algoritmanın ortalama durumu için farklı notasyonlar (örneğin Θ , "tam sınır") kullanılsa da, günlük konuşmada ve teknik mülakatlarda genellikle hepsi kısaca "Big O" olarak anılır.

04 Hangi Görevlerde Kullanılır?

Big O, akademik bir merak konusu olmanın ötesinde, günlük yazılım geliştirme pratiğinde somut kararları etkiler:

Algoritma ve veri yapısı seçimi

Bir listede arama yaparken sıralı diziyi mi, hash tablosunu mu, yoksa dengeli bir ağacı mı kullanmalıyım? Bu tür kararlar, her yapının okuma/yazma/arama işlemlerinin Big O karmaşıklığı karşılaştırılarak verilir.

Yazılım mühendisliği mülakatları

Teknik mülakatlarda adaylardan yazdıkları çözümün zaman ve alan karmaşıklığını ifade etmeleri istenir; bu, adayın çözümünün büyük ölçekte çalışıp çalışmayacağını anlamının standart yoludur.

Veritabanı ve sorgu optimizasyonu

Bir veritabanı sorgusunun tüm tabloyu taraması ($O(n)$) ile bir indeks üzerinden ikili arama yapması ($O(\log n)$) arasındaki fark, milyonlarca satırlık tablolarda saniyeler ile milisaniyeler arasındaki farka dönüşebilir.

Büyük ölçekli sistem tasarımı

Arama motorları, sosyal medya akışları, öneri sistemleri gibi milyonlarca kullanıcıya hizmet veren sistemlerde, küçük bir karmaşıklık farkı (ör. $O(n)$ yerine $O(n^2)$ bir adım) sistemin tamamen kullanılamaz hale gelmesine yol açabilir.

Kaynağı kısıtlı ortamlar

Gömülü sistemler, mobil uygulamalar ve makine öğrenmesi modellerinin eğitim süreçlerinde, sınırlı işlemci ve bellek nedeniyle karmaşıklık analizi doğrudan maliyet ve performans planlamasının bir parçasıdır.

05 Yaygın Karmaşıklık Sınıfları ve Basit Örnekler

Aşağıdaki tablo, en sık karşılaşılan Big O sınıflarını, en hızlıdan en yavaşa doğru, gündelik örnekleriyle birlikte özetler.

Notasyon	Adı	Basit örnek	n = 1.000.000 için yaklaşık adım
$O(1)$	Sabit zaman	Bir dizinin belirli bir indeksindeki elemana doğrudan erişmek	1
$O(\log n)$	Logaritmik zaman	Sıralı bir dizide ikili arama (binary search) yapmak	~20
$O(n)$	Doğrusal zaman	Bir listedeki tüm elemanları tek tek gezmek (doğrusal arama)	1.000.000
$O(n \log n)$	Doğrusal-logaritmik zaman	Merge sort veya heap sort ile bir listeyi sıralamak	~20.000.000
$O(n^2)$	İkinci dereceden (kare) zaman	İç içe iki döngüyle bir listedeki her elemanı diğer her elemanla karşılaştırmak (ör. bubble sort)	1.000.000.000.000
$O(2^n)$	Üstel zaman	Bir kümenin tüm alt kümelerini tek tek üretmek	pratikte hesaplanamaz

Kod üzerinden basit bir örnek

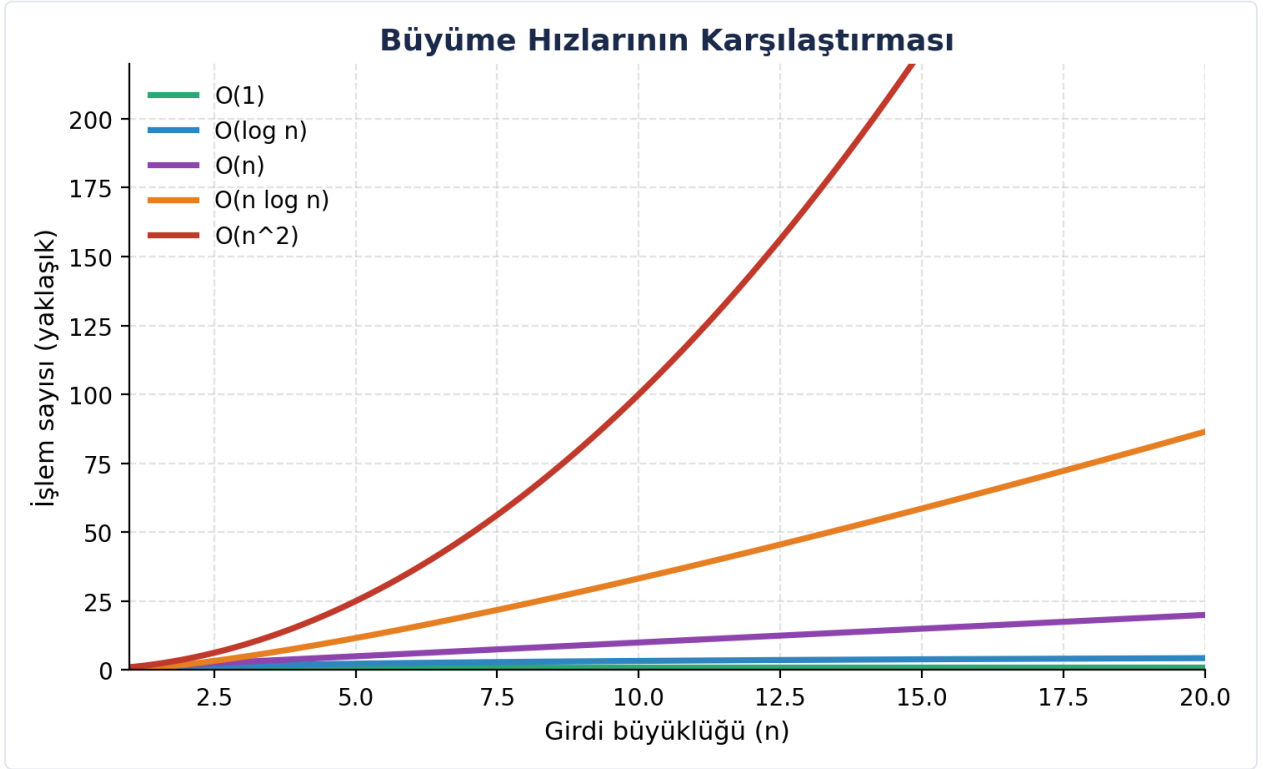
Aşağıdaki iki fonksiyon aynı soruyu (bir listede bir sayı var mı?) farklı karmaşıklıkla cevaplar:

```
# O(n) - doğrusal arama: en kötü durumda listenin tamamı gezilir
def listede_var_mi(liste, hedef):
    for eleman in liste:
        if eleman == hedef:
            return True
    return False

# O(log n) - ikili arama: liste sıralıysa her adımda arama alanı yarıya iner
def ikili_arama(sirali_liste, hedef):
    sol, sag = 0, len(sirali_liste) - 1
    while sol <= sag:
        orta = (sol + sag) // 2
        if sirali_liste[orta] == hedef:
            return True
        elif sirali_liste[orta] < hedef:
            sol = orta + 1
```

```
else:  
    sag = orta - 1  
return False
```

1.000.000 elemanlı sıralı bir listede doğrusal arama en kötü durumda 1.000.000 karşılaştırma yapabilirken, ikili arama aynı listede en fazla yaklaşık 20 karşılaştırmada sonuca ulaşır. Aradaki fark, girdi büyüdükçe katlanarak artar; bu da Big O'nun neden önemli olduğunu somutlaştırır.



Şekil 1. n arttıkça farklı karmaşıklık sınıflarının işlem sayısının nasıl büyüdüğünün karşılaştırması ($n = 1$ ile 20 arası, küçük ölçek). $O(n^2)$ eğrisinin diğerlerinden ne kadar hızlı ayrıldığına dikkat edin.

06 Özet

- **Ne:** Big O, bir algoritmanın girdi büyüdükçe kaynak kullanımının (zaman/bellek) nasıl büyüdüğünü tarif eden matematiksel bir üst sınır gösterimidir.
- **Ne zaman doğdu:** Kökeni 1894'e, Paul Bachmann'a dayanır; Edmund Landau tarafından 1909'da yaygınlaştırılmış, Donald Knuth tarafından 1976'da bilgisayar bilimi için standartlaştırılmıştır.
- **Nerede kullanılır:** Algoritma/veri yapısı seçiminde, teknik mülakatlarda, veritabanı optimizasyonunda, büyük ölçekli sistem tasarımında ve kaynağı kısıtlı ortamlarda.
- **Amaç:** Donanımdan bağımsız, adil ve ölçeklenebilir bir karşılaştırma standardı sunmak.

Bu özet, 5 dakikalık bir sunumun iskeletini oluşturacak şekilde tasarlanmıştır: tanım, tarihçe, amaç, kullanım alanları ve örnekler sırasıyla birer bölüm olarak ele alınabilir.

07 Kaynakça

- **Big O notation, Wikipedia**

Tanım, tarihçe (Bachmann, Landau) ve karmaşıklık sınıflarının genel özeti için kullanıldı.

https://en.wikipedia.org/wiki/Big_O_notation

- **History of Algorithmic Analysis, CS62, Pomona College**

Algoritma analizinin matematikten bilgisayar bilimine geçiş sürecinin tarihsel özeti için kullanıldı.

<https://cs.pomona.edu/classes/cs62/history/bigO/>

- **Donald E. Knuth, "Big Omicron and Big Omega and Big Theta", ACM SIGACT News, Nisan-Haziran 1976**

O, Ω ve Θ notasyonlarının bilgisayar bilimi için standartlaştırıldığı orijinal makale; tarihsel iddiaların (Bachmann 1894, Landau, Hardy-Littlewood 1914) doğrulanması için kullanıldı.

<https://danluu.com/knuth-big-o.pdf>

- **Big-O Complexity Cheat Sheet**

Yaygın veri yapıları ve sıralama algoritmalarının zaman/alan karmaşıklığı tablosunun çapraz kontrolü için kullanıldı.

<https://www.bigocheatsheet.com/>

- **Donald Knuth, "The Art of Computer Programming", Cilt 1 (1968) ve "Analiz Terimi" (Analysis of Algorithms, 1967-68 dönemi)**

Algoritma analizinin bilgisayar biliminde ayrı bir disiplin haline gelme sürecinin arka planı için referans alındı.

https://en.wikipedia.org/wiki/Donald_Knuth

Not: Bu rapordaki tarihler ve isimler, en az iki bağımsız kaynak (Wikipedia, Pomona College ders notları ve Knuth'un 1976 tarihli orijinal makalesi) karşılaştırılarak doğrulanmıştır. Sayısal örnekler ($n = 1.000.000$ için adım sayıları) yaklaşık değerlerdir ve kavramı somutlaştırmak amacıyla hesaplanmıştır.